

FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Daniele Cattaneo

Exam of WED 9 JULY 2025 – Theory Part

LAST NAME (SURNAME) + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. D. CATTANEO

INSTRUCTIONS – READ CAREFULLY:

- The exam is open book: textbooks and personal notes are permitted.
- Pencil writing is permitted, except on the cover page (this sheet).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: lab part 1 h – theory part 2 h.

SCORING AND GRADE ATTRIBUTION:

- The exam is in written form and consists of two parts:

Theory syntax and semantics of languages, divided in four sections: (75% of the final grade)

- 1) regular expressions and finite automata (1 exercise × 10 points)
- 2) syntax analysis and parsing methodologies (1 exercise × 10 points)
- 3) grammar design – translation – semantic analysis (2 exercises × 5 points)
- 4) *bonus* question (1 exercise × 5 points)

Lab compiler design by Flex and Bison (25% of the final grade)

- 1) basic questions (30 points)
- 2) *bonus* question (5 points)

- For the theory part to be valid, the candidate must achieve a grade of at least 5/10 in each of the three sections 1, 2 and 3. If this is not the case, section 4 is not evaluated. Section 4 has no threshold on its own. Correctly answering all the questions in sections 1, 2 and 3 allows the candidate to achieve a grade of 30/30 (but not *cum laude*) for the theory part.
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30 in the basic questions. The bonus question is evaluated only if the threshold is reached.
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- The final grade is the weighted average of the theory part (75 %) and of the lab part (25 %), and must be $\geq 18/30$ (*cum laude* $\geq 32/30$).

1 Regular expressions and finite automata (10 points)

Consider the following regular expression R :

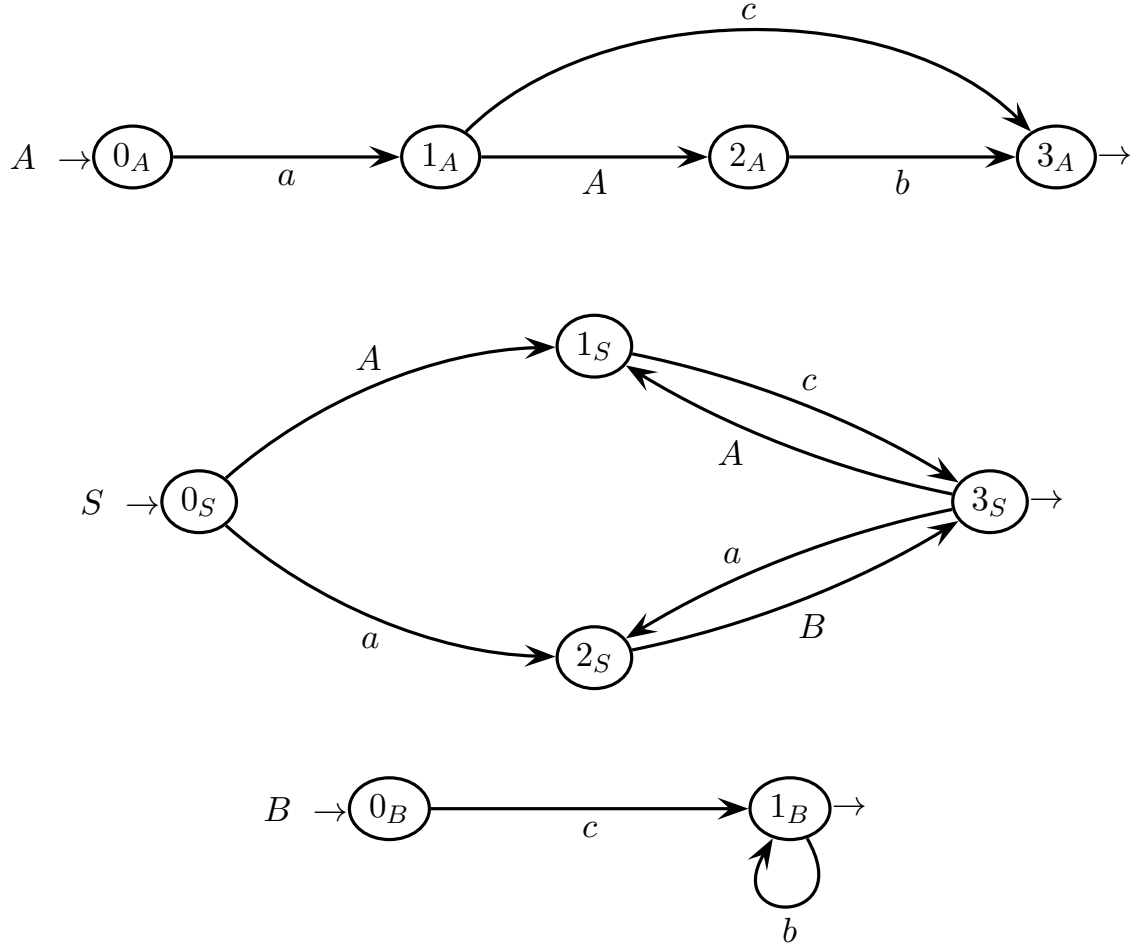
$$R = a b^* a^* a \mid b a^*$$

Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) By using the Berry-Sethi method (BS), find a deterministic finite-state automaton A that recognizes the language $L(R)$.
 - (b) Is the deterministic automaton A minimal? If not, minimize it.
 - (c) Call A_{min} the minimal automaton obtained above (point (b)). By using the Brzozowski-McCluskey method (BMC), find a regular expression R' that generates the language $L(A_{min})$.
 - (d) Find the finite-state automaton that recognizes the mirror language $L(A_{min})^R$. Is this automaton deterministic?
 - (e) Find the finite-state automaton that recognizes the complement language $\overline{L} = \Sigma^* - L(A_{min})$.
-

2 Syntax analysis and parsing methodologies (10 points)

Consider the EBNF grammar G below, represented as a machine network over the three-letter terminal alphabet $\Sigma = \{a, b, c\}$ and the three-symbol nonterminal alphabet $V = \{A, B, S\}$ (axiom S):



Answer the following questions (use the figures / tables / spaces on the next pages):

- Write all the strings of length ≤ 3 (less than or equal to three) that are generated by grammar G , and draw their syntax trees.
- Draw the complete pilot graph of grammar G and show that grammar G is ELR. Briefly justify that the pilot does not have SR, RR or Convergence conflicts.
- Simulate the recognition process of the ELR syntax analyzer of grammar G for the valid string (of length 5) below:

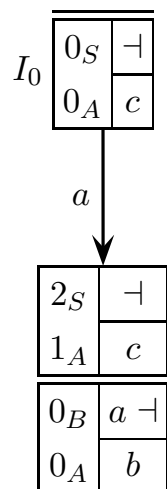
$$a c a c c \in L(G)$$

and show the construction process of the syntax tree of the string.

- Compute all the guide sets on the call arcs and exit arrows of the machine net of grammar G . Are there any guide set conflicts? Justify your answer.
- Say if grammar G is $ELL(k)$ for some $k \geq 1$. Justify your answer.

valid strings of length ≤ 3 and their trees for grammar G – question (a)

complete pilot of grammar G – question (b)



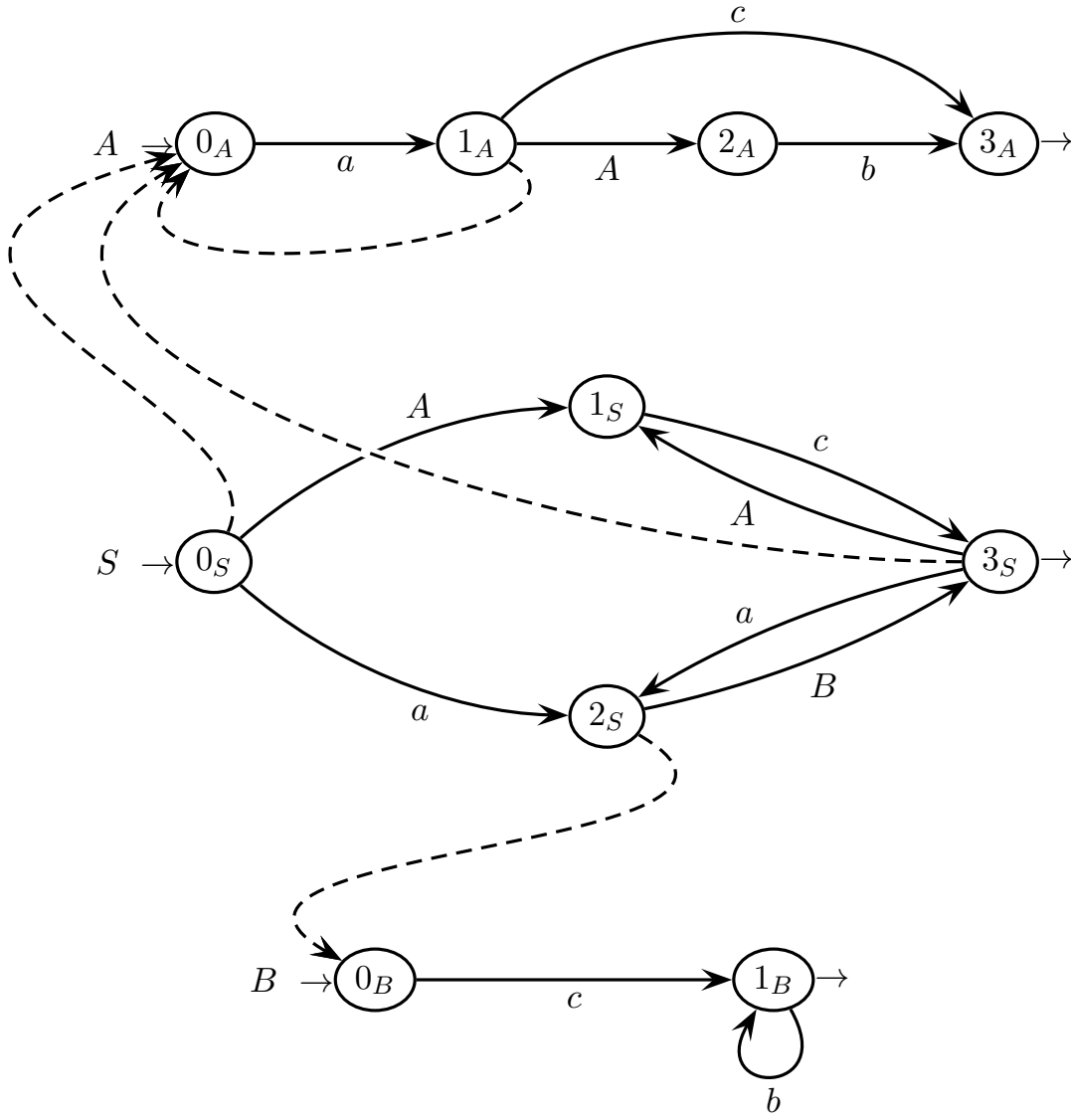
ELR conflict analysis – question (b)

simulation table of the ELR parsing process of string $a c a c c$ – question (c)
(the number of rows is not significant)

[illegible]

construction of the syntax tree – question (c)

guide sets of grammar G – question (d)



ELL conflict analysis – question (d)

ELL analysis for $k \geq 1$ – question (c)

3 Grammar design – translation – semantic analysis (5 + 5 points)

1. Consider a language for configuration files, consisting in a list of definitions of **configuration variables**. Configuration variables are booleans that can be turned *on* or *off*. A basic example of a configuration file is the following:

```
FAST_DEMOLITION = ON;
PLANT_TREES = OFF;
```

FAST_DEMOLITION and PLANT_TREES are examples of variable identifiers, and are symbolized by the terminal `id`. Each definition consists of the identifier `id`, followed by an equals sign “=”, the configuration value `ON` or `OFF`, and a terminating semicolon “;”. A file cannot be empty.

On the left side of an equals sign, it is also possible to specify multiple variables, separated by commas “,”, to set all of them at the same time:

```
FAST_DEMOLITION, USE_MAGIC_MALLET = ON;
USE_CHEAP_HELMETS, USE_CHEAP_RESTRAINTS, USE_CHEAP_TRUSSES = OFF;
```

On the right side of an equals sign, instead of a single literal `ON` or `OFF`, it is possible to have an arbitrary expression that combines other variables by using the logical operators *and*, *or* and *not*:

```
HIRE_SORCERER = (FAST_DEMOLITION & USE_MAGIC_MALLET) & !USE_CHEAP_HELMETS;
MAGIC_TREE_GROWTH = HIRE_SORCERER | PLANT_TREES;
```

The rules for the expressions are the following:

- the *and* operator appears as “&”, *or* appears as “|”, and *not* appears as “!”
- operator *and* is left-associative and *or* is right-associative
- operator *not* has a higher priority than *and*
- operator *and* has a higher priority than *or*
- subexpressions can be arbitrarily grouped in parentheses “()”

Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Write an EBNF non-ambiguous grammar (axiom `FILE`) that generates the language described.
- (b) Draw the syntax tree of the sample configuration file shown below:

```
FAST_DEMOLITION, USE_MAGIC_MALLET = OFF;
HIRE_SORCERER = FAST_DEMOLITION & !USE_MAGIC_MALLET;
```

- (c) Extend the grammar to add the possibility of arbitrarily grouping non-empty lists of variable definitions within (possibly nested) curly brackets “{ }”:

```
PLANT_TREES = OFF;
{
    USE_CHEAP_HELMETS, USE_CHEAP_RESTRAINTS, USE_CHEAP_TRUSSES = OFF;
    {
        HIRE_SORCERER = (FAST_DEMOLITION & USE_MAGIC_MALLET) & !USE_CHEAP_HELMETS;
    }
    FAST_DEMOLITION, USE_MAGIC_MALLET = ON;
}
```


2. Consider the language of the expressions with a binary operator $+$, where each (sub)expression can be either of *prefix* type (Polish) or of *postfix* type (reverse Polish). Prefix-type and postfix-type subexpressions are distinguished by being enclosed in round parentheses “()” and square parentheses “[]”, respectively; the whole expression itself is enclosed in parentheses.

This language is generated by the following source BNF grammar G , over the three-symbol nonterminal alphabet $V = \{S, A, B\}$ and the six-letter terminal alphabet $\Sigma = \{+, n, “(”, “)” “[”, “]”\}$ (axiom S); see also the sample valid string aside:

$$G \left\{ \begin{array}{l} 1: S \rightarrow “(” + A A “)” \\ 2: S \rightarrow “[” B B + “]” \\ 3: S \rightarrow n \\ \hline 4: A \rightarrow + A A \\ 5: A \rightarrow n \\ 6: A \rightarrow “[” B B + “]” \\ \hline 7: B \rightarrow B B + \\ 8: B \rightarrow n \\ 9: B \rightarrow “(” + A A “)” \end{array} \right. \quad \begin{array}{l} \text{sample string of the source language } L(G) \\ \\ \\ \\ \\ (+ + n n [n n +]) \end{array}$$

Consider now the language of the *infix* expressions with a binary operator $+$. These expressions are specified so that *each* operator (including the top-level one) is enclosed in round parentheses. A (BNF) grammar G' that generates this language, over the one-symbol nonterminal alphabet $V' = \{S\}$ and the four-letter terminal alphabet $\Sigma' = \{+, n, “(”, “)”\}$, is (axiom S):

$$G' \left\{ \begin{array}{l} S \rightarrow “(” S + S “)” \\ S \rightarrow n \end{array} \right. \quad \begin{array}{l} \text{sample string of language } L(G') \\ ((n + n) + (n + n)) \end{array}$$

Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Write a BNF translation grammar or scheme G^τ , with source grammar G , that syntactically translates a string of the source language $L(G)$ to the destination (target) language $L(G')$. For instance (the double brackets “[]” just delimit the expression given as argument to translation τ , and disappear from the result):

$$\tau \llbracket (+ + n n [n n +]) \rrbracket = ((n + n) + (n + n))$$

Caveat: grammar G' is *not* the destination (target) grammar of G^τ ; it only serves to clearly specify the destination (target) language of G^τ .

- (b) Draw the translation tree of grammar G^τ for the sample translation shown above (point (a)).
- (c) Consider the language of infix binary expressions that admits both round parentheses and square parentheses. Modify the previously defined translation grammar or scheme G^τ to retain the kind of parenthesis that encloses each expression in the translation (call θ the new syntactic translation and G^θ the new BNF translation grammar or scheme). For instance:

$$\theta \llbracket (+ + n n [n n +]) \rrbracket = ((n + n) + [n + n])$$

You can show only the new or modified rules.

- (d) Are the translations τ and θ invertible? Explain your answer.

free space for continuation or notes – any question

free space for continuation or notes – any question

4 Bonus question (5 points)

Consider a language L of nested lists. Each (sub)list (possibly empty) is enclosed in parentheses “()” and contains elements schematized by a letter e , without separators. The outermost list is always enclosed in parentheses. The BNF grammar G below generates language L (axiom S):

$$G \left\{ \begin{array}{l} 1: S \rightarrow (E) \\ 2: E \rightarrow \varepsilon \\ 3: E \rightarrow e E \\ 4: E \rightarrow (E) E \end{array} \right. \quad \begin{array}{l} \text{sample string of language } L = L(G) \\ (e (e e e) e ()) \end{array}$$

We want to define an attribute grammar, based on the syntactic support G , that computes a semantic translation τ . This translation transforms a nested list into a nested list with the same parenthetical structure, but without elements e . For each (sub)list:

- the opening parenthesis is immediately followed by the nesting depth (starting from the outermost list) of the contents of the (sub)list
- the closing parenthesis is immediately preceded by the total number of elements e contained (at any depth level) in the (sub)list

Here are some sample translations (the bracket pair “[]” just delimits the argument of τ):

$$\begin{aligned} \tau \llbracket () \rrbracket &= (1 \ 0) & \tau \llbracket (e) \rrbracket &= (1 \ 1) & \tau \llbracket (e e) \rrbracket &= (1 \ 2) & \tau \llbracket (e (e)) \rrbracket &= (1 \ (2 \ 1) \ 2) \\ \tau \llbracket (e (e e e) e ()) \rrbracket &= (1 \ (2 \ 3) \ (2 \ 0) \ 5) & & // \text{ sample translation for the tree} \end{aligned}$$

To this purpose, two attributes d and n (below) and their semantic functions (next pages) are given:

nonterminal	name	type	domain	meaning
E	d	right	integer ≥ 1	nesting <i>depth</i> of the contents of a (sub)list
E	n	left	integer ≥ 0	total <i>number</i> of elements e in a (sub)list

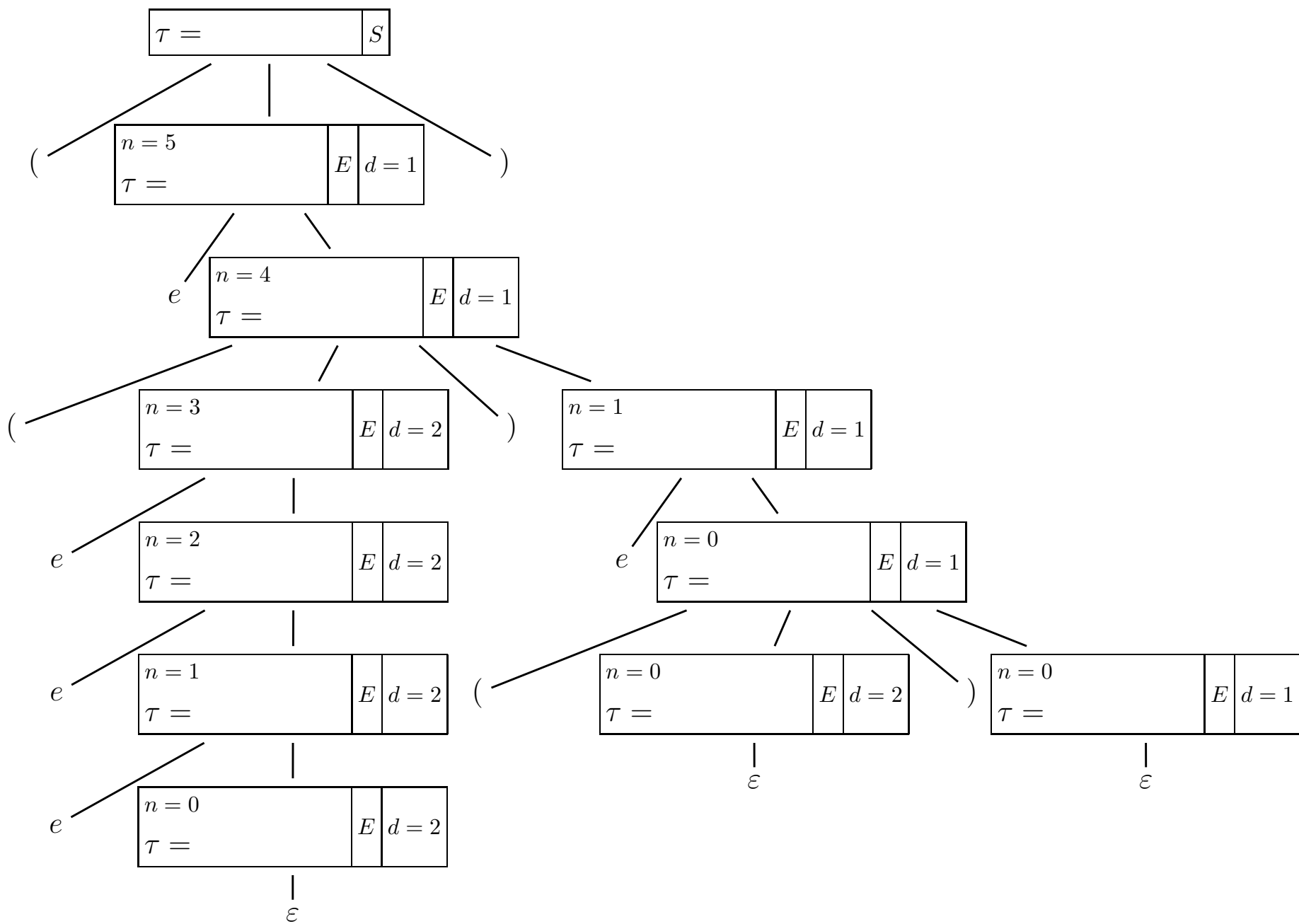
Answer the following questions (use the figures / tables / spaces on the next pages):

- Complete the attribute grammar already sketched (next page), by adding to it a left (synthesized) attribute τ , of value string, associated to both nonterminals S and E . The attribute τ provides, at the tree root S , the translation τ of the entire original string (you can use the operator “.” to concatenate strings and numbers). Make sure that the complete attribute grammar is one-sweep.
- Decorate the translation tree of the (longest) sample string above with the values of attribute τ .
- Is the complete attribute grammar of type L ? Explain why. If so, write the recursive-descent integrated syntactic-semantic procedure of the nonterminal S .

attribute grammar to be completed – question (a)

nonterminal	name	type	domain	meaning
S, E	τ	left	string	translation

#	<i>syntax</i>	<i>semantics</i>
1:	$S_0 \rightarrow (E_1)$	$d_1 := 1$ $\tau_0 :=$
2:	$E_0 \rightarrow \varepsilon$	$n_0 := 0$ $\tau_0 :=$
3:	$E_0 \rightarrow e E_1$	$d_1 := d_0$ $n_0 := n_1 + 1$ $\tau_0 :=$
4:	$E_0 \rightarrow (E_1) E_2$	$d_1 := d_0 + 1$ $d_2 := d_0$ $n_0 := n_1 + n_2$ $\tau_0 :=$



recursive-descent integrated syntactic-semantic procedure of S – question (c)

free space for continuation or notes – any question